How to initialize super class properties (stack) through sub class constructor :

* In order to initialize super class properties we should use super keyword.

super keyword :

* It is used to **access the member OR memory of super class**

* It is used to access the member OR memory of immediate super class. If we have 3 classes in multilevel inheritance i.e. A -> B -> C

* From class C, the can only access the member of B class. There is no provision in java, to access the member of class A directly from class C.

* Just like this keyword, super keyword also represents non-static member as, We cannot use super keyword from the static context.

* super keyword, We can use 3 ways in java :

1) To call the super class non-static field (Variable Hiding)
2) To call the super class non-static method (Method Overriding)
3) To call the super class constructor (Construction Chaining).

1) To call the super class non-static field (Variable Hiding) :

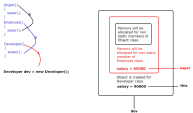
Variable hiding occurs when a subclass defines a variable with the same name as a variable in its superclass. Variable hiding, both sub class and super class field **name** same then it is called variable hiding, here sub class non-static field **hides** super class non static field.

In order to access super class non static field i.e. super class memory, we should use super keyword as shown in the program.

VariableHidingDemo.java

```
package com.rel.super_keyword;

class Employee
{
    protected double salary = 15000;

}

class Developer extends Employee
{
    protected double salary = 30000;  //variable hiding

    public void printSalary()
    {
        IO.println("Developer Salary is :"+this.salary);
        IO.println("Employee Salary is :"+super.salary);
    }
}

public class VariableHidingDemo
{
    public static void main(String[] args)
    {
        Developer developer = new Developer();
        developer.printSalary();
    }
}
```

Diagram for the above program :



Important Points :

* If we create an object for sub class then internally super class constructor will execute and **only memory is allocated for non static members.**

* IN ALL CASES JVM COMPILER WILL BIND fields and methods by using **current reference Frame** (this) **but in case of static field or static type Developer no compiler will always search variable and method in current class (based on the current reference)**

* JVM will always execute the method not based on the current reference but actually based on the current object.

non static field initialization + class class loading program :

```
package com.rel.super_keyword;
class Employee
{
    protected double salary = 15000;

    public Employee()
    {
        printSalary();
    }

    public void printSalary()
    {
        IO.println(salary);
    }
}

class Developer extends Employee
{
    protected double salary = 30000;  //variable hiding

    public void printSalary()
    {
        IO.println(salary);
        IO.println(super.salary);
    }
}

public class VariableHidingDemo
{
    public static void main(String[] args)
    {
        Developer developer = new Developer();
    }
}
```

Output : 0.0 , 0.0300.0

2) To access super class non static method (Method Overriding):

If the super class non static method name and sub class non static method name both are same (Method Overriding), then sub class non static method **hides** super class non static method. So, when it is executed (called in test), if we want to call super class method from sub class method body then, we should use super keyword as shown in the program.

Program

```
package com.rel.super_keyword;

class Alpha
{
    public void show()
    {
        IO.println("show method of Alpha class");
    }
}

class Beta extends Alpha
{
    @Override
    public void show()
    {
        super.show();
        IO.println("show method of Beta class");
    }
}

public class MethodOverriding
{
    public static void main(String[] args)
    {
        Beta beta = new Beta();
        beta.show();
    }
}
```

Another Program :

```
package com.rel.super_keyword;

class Alpha
{
    @Override
    public String toString()
    {
        return "Alpha";
    }
}

class Beta extends Alpha
{
    @Override
    public String toString()
    {
        return "Beta";
    }
}

class Gamma extends Beta
{
    @Override
    public String toString()
    {
        return "Gamma";
    }
}

public class MethodOverriding
{
    public static void main(String[] args)
    {
        Gamma g = new Gamma();
        IO.println(g.toString());
    }
}
```

Java compiler will always search based on the current reference and JVM will always execute based on the current object.

To call the super class constructor :

* In order to call the super class constructor, The first line of any constructor is reserved for super()

* From the first line of the constructor, we can also call current class overloaded constructor by using this()

* So, The Constructor is

a) super() : It is used to call **super** class no argument/default constructor (Added by java compiler)

b) super(10) : It is used to call parameterized constructor of **super** class (Written by user)

c) this() : It is used to call **current** class no argument constructor (Written by user)

d) this(10) : It is used to call parameterized constructor of **current** class (Written by user)

WAP to call super class no argument constructor : (super())

```
package com.rel.super_keyword;

class Alpha
{
    public Alpha()
    {
        IO.println("No argument constructor of Alpha class");
    }

}

class Beta extends Alpha
{
    public Beta()
    {
        IO.println("No argument constructor of Beta class");
    }
}

class Gamma extends Beta
{
    public Gamma()
    {
        IO.println("No argument constructor of Gamma class");
    }

}

public class CallingNoArgument
{
    public static void main(String[] args)
    {
        new Gamma();
    }
}
```

Calling parameterized constructor of super class to initialize the non static fields of super class :

```
package com.rel.super_keyword;

class Vehicle
{
    protected String name;

    public Vehicle(String name)
    {
        super();
        this.name = name;
    }

}

class Car extends Vehicle
{
    public Car(String name)
    {
        super(name);
    }

    @Override
    public String toString()
    {
        return "Car [name= " + name + "]";
    }
}

public class CallingParameterizedConstructorCar {

    public static void main(String[] args) {
        String carname = IO.readln("Enter the name of the car ");
        Car car = new Car(carname);
        IO.println(car);
    }

}
```